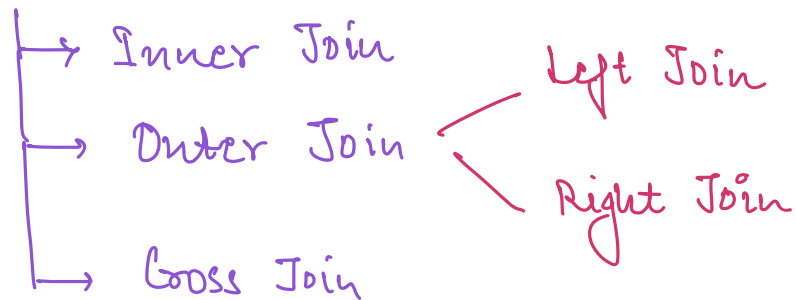


→ JOINS



→ Aggregate Queries.

⇒

Students

St-id	name	email	batch-id
1	Harold	h@—	2
2	Kalpesh	—	4
3	Sagarika	—	3
4	Sahil	—	1
5	Parvathi	—	2

batches

batch-id	b-name	...
1	SDE	
2	AIML	
3	DS	
4	Agentic	

Q: Give me the students who belongs to batch-id = 2

→ Select *
from Students
where batch_id = 2;

Q: fetch student details for email = —;

→ Select *
from Students
where email = —;

Q: Give me student names along with their batch names.

→ JOIN.

Students

St-id	name	email	batch-id
1	Harold	h@—	2
2	Kalpesh	—	4
3	Sagarika	—	3
4	Sahil	—	1
5	Parvesh	—	2
6	—	—	Null

batches

batch-id	b-name	...
1	SDE	
2	AIML	
3	DS	
4	Agentie	
5	—	

JOIN

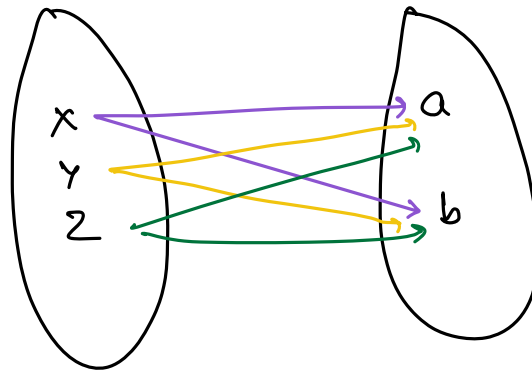


Cross Join

St-id	name	email	b-id	b-id	name - - -
1	Harold	—	2	1	SDE
1	Harold	—	2	2	AIML
1	Harold	—	2	3	DS
1	Harold	—	2	4	Agentie AI

Table-1

Table-2



Cross
JOIN.

Select *
From students
JOIN batches

(OR)

Select *
From students, batches

Inner
Join.

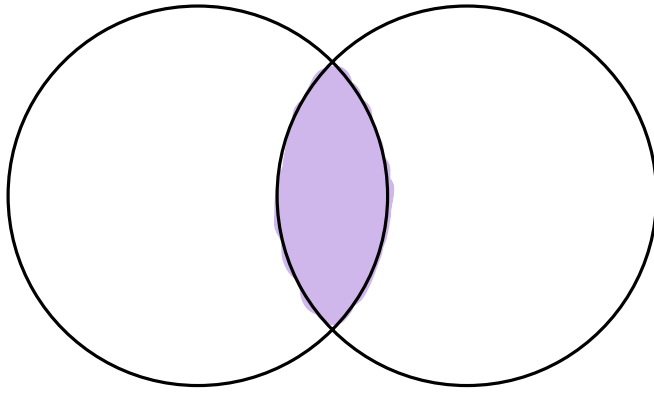
Select *
From students st
JOIN batches b

ON st.batch_id = b.batch_id

st-id	name	email	b-id	b-id	name . . .
1	Harold	—	2	2	AIML
2	K	—	4	4	Agentix
3	S	—	3	3	DS
4	S	—	1	1	SDE
5	P	—	2	2	AIML

Students

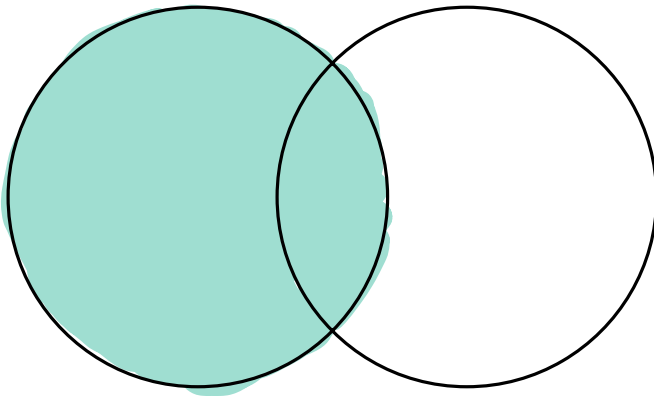
batches



INNER JOIN.

Students

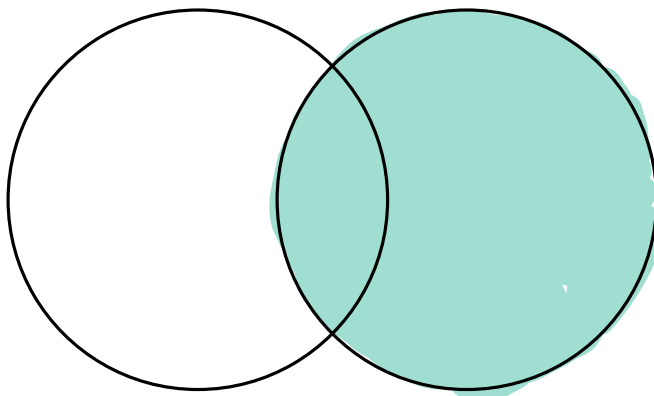
batches



LEFT JOIN

Students

batches



RIGHT JOIN.

SELF JOIN.

employees.

emp-id	name	email	manager-id
1	ABC	—	4
2	XY2	—	4
3	PQA	—	5
4	MNO	—	5
5	DEF	—	

→ Get the employee name along with their manager name.

employees (e1)

emp-id	name	email	manager-id
1	ABC	—	4
2	XY2	—	4
3	PQR	—	5
4	MNO	—	5
5	DEF	—	

employees (e2)

emp-id	name	email	manager-id
1	ABC	—	4
2	XY2	—	4
3	PQR	—	5
4	MNO	—	5
5	DEF	—	

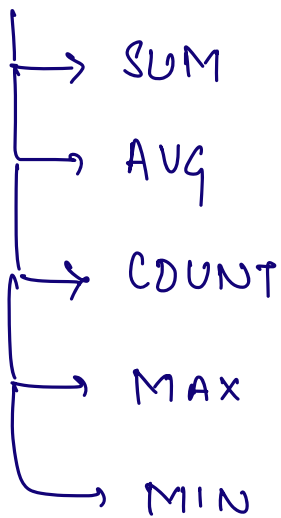
Select *

from employees e1

join employees e2

ON e1.manager-id = e2.emp-id

AGGREGATE FUNCTIONS.



Students

id	name	batch-id	marks
1	A	4	70
2	B	3	80
3	C	2	60
4	D	1	40
5	E	NULL	50

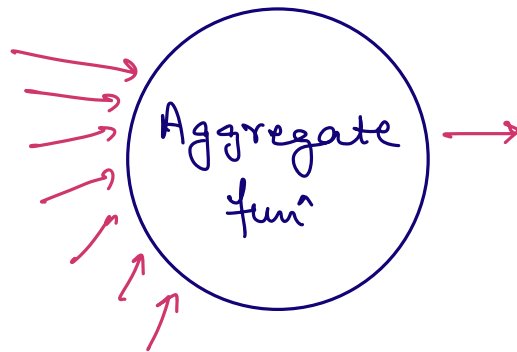
Q: Find the count of batches in students table

Select COUNT(batch-id)
from students } ⇒ ④

Count fun[^] takes only NON
NULL values.

AVG(marks)

SUM(marks)



#

Students

id	name	batch-id	marks
1	A	2	80
2	B	3	80
3	C	1	90
4	D	2	80
5	E	3	90
6	F	1	60

batches

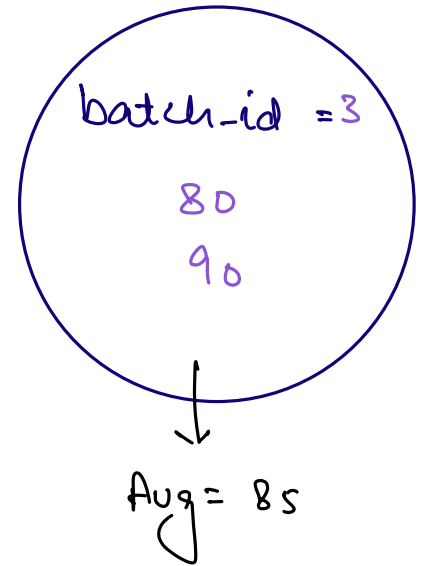
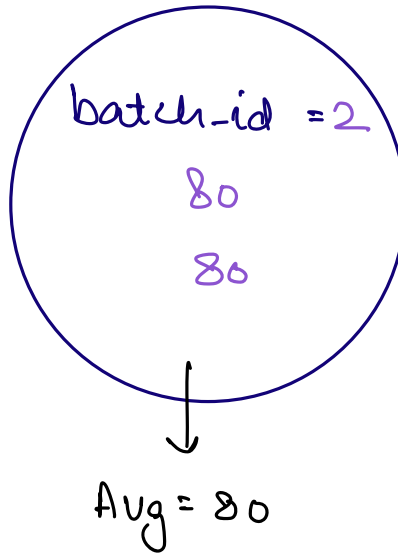
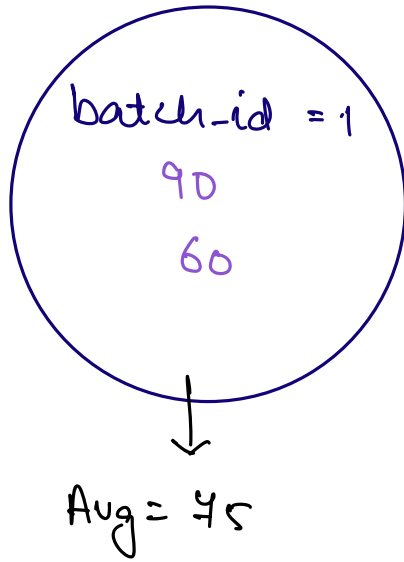
batch-id	name
1	b-A
2	b-B
3	b-C

Q. Get the avg marks of all the students.

select avg(marks)
from students;

⇒ 80

Q. Get the avg marks of every batch.



→ Group the students based on batch-id.

Select batch-id, avg(marks), ~~name~~
from students
group by batch-id;

batch-id	avg-PSP
1	75
2	80
3	85

Students

id	name	batch-id	marks
1	A	2	80
2	B	3	80
3	C	1	90
4	D	2	80
5	E	3	90
6	F	1	60

batches

batch-id	name
1	b-A
2	b-B
3	b-C

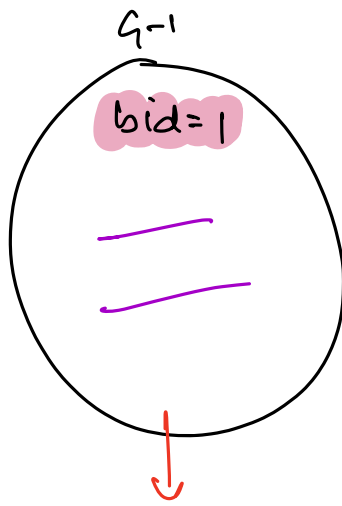
Q: Get the avg psp of each batch along with their batch name

id	name	batch-id	marks	batch-id	batch-name
1	A	2	80	2	b-B
2	B	3	80	3	b-C
3	C	1	90	1	b-A
4	D	2	80	2	b-B
5	E	3	90	3	b-C
6	F	1	60	1	b-A

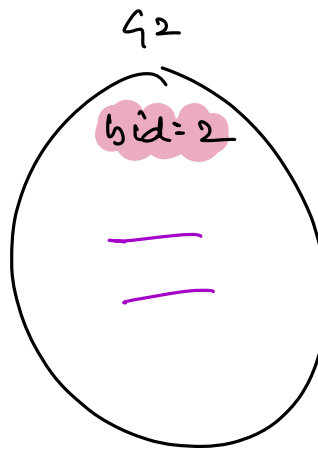
```

Select batch-id, avg(marks), batch-name
from students s
join batches b
ON s.batch-id = b.id
Group By batch-id, batch-name

```

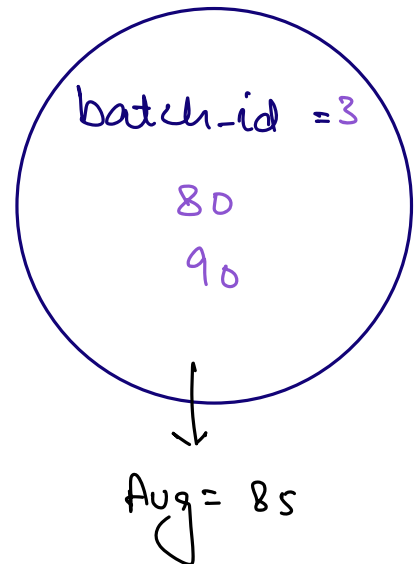
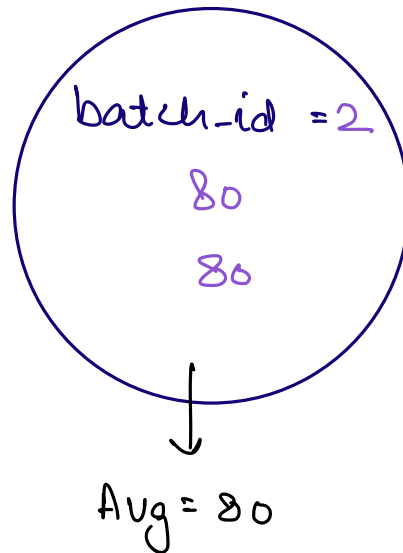
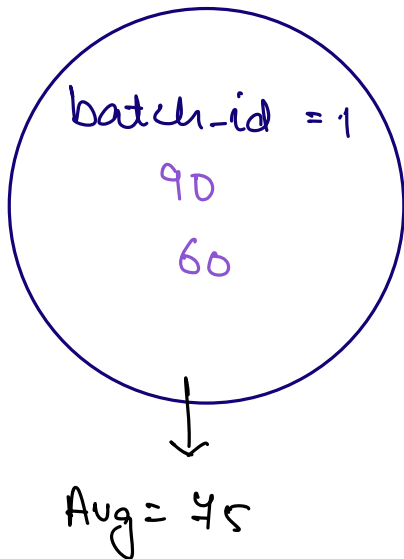


Avg | Count | Sum |
Max | Min |



Q. Get the avg marks of every batch only if
avg marks > 80

→ filter groups.



→ Group the students based on batch-id.

X {
Select batch-id, avg(marks)
from students
group by batch-id
WHERE avg(marks) > 80;
 ↓ filter rows.

X
NOT
CORRECT.

Select batch-id, avg(marks) as avg-marks
from students

group by batch-id

HAVING avg-marks > 80
↓

filter groups.

-- JOINS ---

```
select * from students;
```

```
select * from batches;
```

```
INSERT INTO students VALUES (1, 'Harold', 'harold@gmail.com', 20, 2);
INSERT INTO students VALUES (2, 'Kalpesh', 'Kalpesh@gmail.com', 21, 3);
INSERT INTO students VALUES (3, 'Sagarika', 'Sagarika@gmail.com', 22, 1);
INSERT INTO students VALUES (4, 'Parmesh', 'Parmesh@gmail.com', 23, 4);
INSERT INTO students VALUES (5, 'Shelin', 'Shelin@gmail.com', 23, 2);
INSERT INTO students VALUES (6, 'Jyotirmay', 'Jyotirmay@gmail.com', 21, null);
```

```
INSERT INTO batches VALUES (5, 'Data Science');
```

-- Q: Get the name of all the students with their batch names.

-- INNER JOIN

```
select s.name, b.batch_name
from students s
join batches b
ON s.batch_id = b.batch_id;
```

-- LEFT JOIN - In left join, we get all the data from the left table and the matching data from right table.

```
select s.name, b.batch_name
from students s
left join batches b
ON s.batch_id = b.batch_id;
```

-- RIGHT JOIN

```
select s.name, b.batch_name
from students s
right join batches b
ON s.batch_id = b.batch_id;
```

-- FULL JOIN: MySQL doesn't support FULL JOIN directly

```
create table IF NOT EXISTS employees (
    emp_id integer unique not null,
    name varchar(50),
    manager_id integer
);
```

```
INSERT INTO employees (emp_id, name) VALUES (1, 'Harold');
INSERT INTO employees (emp_id, name) VALUES (2, 'Kalpesh');
INSERT INTO employees (emp_id, name) VALUES (3, 'Shelin');
INSERT INTO employees (emp_id, name) VALUES (4, 'Parmesh');
INSERT INTO employees (emp_id, name) VALUES (5, 'Jyotirmay');
INSERT INTO employees (emp_id, name) VALUES (6, 'Mukund');
```

```
select * from employees;
```

```
update employees  
set manager_id = 5  
where emp_id = 6;
```

```
-- get the employee name and their manager's name  
-- SELF JOIN  
select e1.name as employee_name, e2.name as manager_name  
from employees e1  
join employees e2  
on e1.manager_id = e2.emp_id;
```